

LAPORAN TUGAS 2

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mochamad Rio Aprilian (2410651040)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan sistem operasi modern telah mentransformasi paradigma komputasi dari eksekusi tunggal menuju konsep multitasking dan konkurensi, dimana manajemen proses dan komunikasi antar proses menjadi fondasi fundamental. Dalam arsitektur kontemporer, setiap proses berjalan dalam ruang memori terisolasi untuk menjamin stabilitas sistem, namun isolasi ini menciptakan tantangan kompleks dalam koordinasi dan pertukaran data. Praktikum ini mengeksplorasi implementasi process management dan inter-process communication (IPC) melalui dua skenario nyata: sistem manajemen proses paralel dan aplikasi chat real-time. Konsep forking, piping, shared memory, dan semaphore synchronization dipelajari secara mendalam untuk memahami bagaimana proses-proses independen dapat berkolaborasi menyelesaikan tugas kompleks. Pemahaman tentang trade-off antara performa, kompleksitas implementasi, dan keamanan dalam pemilihan mekanisme IPC menjadi krusial dalam pengembangan distributed systems dan aplikasi multithreading. Melalui eksperimen langsung, praktikum ini tidak hanya menguji teori-teori fundamental sistem operasi tetapi juga memberikan insight praktis tentang optimalisasi resource management dalam environment komputasi paralel.

1.2 RUMUSAN MASALAH

1. Bagaimana implementasi process management menggunakan `fork()`, `pipe()`, dan `wait()` untuk menciptakan sistem komputasi paralel yang efisien?
2. Apa mekanisme IPC yang tepat untuk membangun aplikasi chat real-time dengan konsekuensi data consistency dan race condition prevention?
3. Bagaimana performa dan kompleksitas implementasi antara mekanisme Pipes dan Shared Memory dalam skenario komunikasi proses yang berbeda?

1.3 TUJUAN

1. Menganalisis dan mengimplementasikan sistem manajemen proses dengan parallel task execution dan sinkronisasi parent-child process

2. Mengevaluasi efektivitas mekanisme Shared Memory dan Semaphore dalam membangun aplikasi komunikasi real-time antar proses
3. Membandingkan karakteristik performa, keamanan, dan kompleksitas antara berbagai metode IPC untuk menentukan solusi optimal berdasarkan kebutuhan aplikasi

BAB II

PEMBAHASAN

2.1 PRAKTIK TUGAS 1

2.1.1 Sistem Manajemen Proses

Program sistem manajemen proses ini merepresentasikan implementasi konkurensi dan koordinasi antar proses dalam sistem operasi melalui teknik fork-exec-wait pattern. Kode tersebut menginisiasi tiga child processes yang dieksekusi paralel, masing-masing melakukan tugas komputasi berbeda: pengurutan array dengan algoritma bubble sort, pencarian elemen menggunakan linear search, serta kalkulasi statistik dasar seperti nilai rata-rata dan ekstrem. Parent process berfungsi sebagai coordinator yang mengatur eksekusi dengan membuat multiple pipes untuk komunikasi antar proses, mengumpulkan hasil komputasi, serta melakukan sinkronisasi menggunakan waitpid() untuk mencegah race conditions. Setiap proses child mengukur waktu eksekusi dengan precision microsecond melalui gettimeofday(), menyimpan hasil dalam struktur data terdefinisi, dan mengirimkannya kembali ke parent melalui mekanisme pipe IPC. Parent process kemudian mengkonsolidasi seluruh hasil termasuk PID, execution time, dan exit status untuk ditampilkan dalam format terstruktur, sekaligus menghitung total waktu eksekusi sistem. Implementasi ini mendemonstrarkan prinsip process isolation, resource sharing, dan inter-process communication yang menjadi fondasi sistem operasi modern, dengan optimasi melalui paralelisasi tugas untuk meningkatkan throughput sistem secara signifikan.

```

rio_regex@DESKTOP-NVPMHQ:~$ touch management_process.c
rio_regex@DESKTOP-NVPMHQ:~$ nano management_process.c
rio_regex@DESKTOP-NVPMHQ:~$ gcc management_process.c -o management_process
rio_regex@DESKTOP-NVPMHQ:~$ ./management_process
=== SISTEM MANAJEMEN PROSES SEDERHANA ===
Parent Process PID: 434

Process 435 (Sorting): Mengurutkan array...
Process 435 (Sorting): Array telah terurut dengan benar
Process 436 (Searching): Mencari nilai dalam array...
Process 436 (Searching): Nilai 42 ditemukan pada index 39

=== PARENT PROCESS MENUNGGU CHILD SELESAI ===
Process 437 (Calculation): Menghitung statistik...
Process 437 (Calculation): Sum=47684, Min=11, Max=996, Average=476.84

=== HASIL EKSEKUSI SEMUA PROSES ===

```

PID	Task	Time (s)	Status
435	Sorting Array	0.000088	Success
436	Searching Value	0.000080	Success
437	Calculation Stats	0.000121	Success

```

=== STATISTIK KESELURUHAN ===
Total processes: 3
Total execution time: 0.000289 seconds
Parent Process PID: 434
Semua child processes telah selesai dieksekusi!
rio_regex@DESKTOP-NVPMHQ:~$ |

```

2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

A. Proses Kompilasi:

- **Pembuatan File Source Code** - Membuat file `management_process.c` menggunakan text editor `nano`
- **Kompilasi dengan GCC** - Menjalankan perintah `gcc management_process.c -o management_process` untuk mengkompilasi kode C menjadi executable
- **Generasi Object Code** - Compiler melakukan parsing, preprocessing, dan translating kode sumber menjadi bahasa mesin
- **Linking Process** - GCC menghubungkan object code dengan library sistem yang diperlukan (seperti `libc` untuk fungsi sistem)

B. Proses Eksekusi:

- **Execution Launch** - Menjalankan program dengan perintah `./management_process`
- **Process Initialization** - Sistem operasi memuat executable ke memory dan membuat proses parent utama (PID 434)
- **Array Generation** - Program menghasilkan array acak berisi 100 elemen untuk diproses
- **Pipe Creation** - Membuat multiple pipes untuk komunikasi inter-process communication
- **Forking Process** - Parent process melakukan `fork()` sebanyak 3 kali untuk membuat child processes
- **Task Distribution** - Setiap child process menjalankan tugas spesifik:
 - **Process 435:** Sorting array dengan bubble sort algorithm
 - **Process 436:** Searching value menggunakan linear search
 - **Process 437:** Statistical calculation (sum, min, max, average)
- **Parallel Execution** - Ketiga child processes berjalan secara paralel dan independen
- **Result Collection** - Parent process mengumpulkan hasil melalui pipes dan `waitpid()`
- **Performance Benchmarking** - Sistem mengukur execution time setiap proses menggunakan `gettimeofday()`

- **Output Display** - Menampilkan hasil eksekusi dalam format terstruktur dengan statistik lengkap

C. Process Termination:

- **Child Process Exit** - Setiap child process mengirim hasil dan melakukan exit(0)
- **Parent Cleanup** - Parent process menunggu semua child selesai dengan waitpid()
- **Resource Release** - Semua pipes dan memory resources dibersihkan
- **Program Completion** - Sistem menampilkan summary statistik dan mengakhiri eksekusi

D.KODE C:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <sys/time.h>

#include <time.h>

#include <string.h>

#define ARRAY_SIZE 100

#define NUM_PROCESSES 3

// Struktur untuk menyimpan hasil proses

typedef struct {

    pid_t pid;

    char task_name[50];

    double execution_time;

    int status;

    char result[100];

} ProcessResult;

// Fungsi untuk menghasilkan array acak

void generate_random_array(int arr[], int size) {

    for (int i = 0; i < size; i++) {

        arr[i] = rand() % 1000;

    }

}
```

```

}

// Fungsi untuk mengukur waktu eksekusi
double get_current_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

// Tugas 1: Sorting (Bubble Sort)
void task_sorting(int arr[], int size) {
    printf("Process %d (Sorting): Mengurutkan array...\n", getpid());
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    // Verifikasi hasil sorting
    int sorted = 1;
    for (int i = 0; i < size - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            sorted = 0;
            break;
        }
    }

    printf("Process %d (Sorting): Array %s terurut dengan benar\n",
        getpid(), sorted ? "telah" : "tidak");
}

```

// Tugas 2: Searching (Linear Search)

```
void task_searching(int arr[], int size) {  
    printf("Process %d (Searching): Mencari nilai dalam array...\n", getpid());  
    int target = 42; // Nilai yang dicari  
    int found = 0;  
    int position = -1;  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == target) {  
            found = 1;  
            position = i;  
            break;  
        }  
    }  
    if (found) {  
        printf("Process %d (Searching): Nilai %d ditemukan pada index %d\n",  
            getpid(), target, position);  
    } else {  
        printf("Process %d (Searching): Nilai %d tidak ditemukan\n",  
            getpid(), target);  
    }  
}
```

// Tugas 3: Calculation (Menghitung statistik)

```
void task_calculation(int arr[], int size) {  
    printf("Process %d (Calculation): Menghitung statistik...\n", getpid());  
    int sum = 0;  
    int min = arr[0];  
    int max = arr[0];  
    for (int i = 0; i < size; i++) {  
        sum += arr[i];  
        if (arr[i] < min) min = arr[i];  
    }  
}
```

```

    if (arr[i] > max) max = arr[i];
}

double average = (double)sum / size;
printf("Process %d (Calculation): Sum=%d, Min=%d, Max=%d, Average=%.2f\n",
getpid(), sum, min, max, average);
}

int main() {
printf("=== SISTEM MANAJEMEN PROSES SEDERHANA ===\n");
printf("Parent Process PID: %d\n\n", getpid());
int array[ARRAY_SIZE];
generate_random_array(array, ARRAY_SIZE);
pid_t pids[NUM_PROCESSES];
int pipefd[NUM_PROCESSES][2];
ProcessResult results[NUM_PROCESSES];

// Buat pipes untuk komunikasi
for (int i = 0; i < NUM_PROCESSES; i++) {
if (pipe(pipefd[i]) == -1) {
perror("Pipe creation failed");
exit(1);
}
}

// Buat multiple child processes
for (int i = 0; i < NUM_PROCESSES; i++) {
pids[i] = fork();
if (pids[i] == -1) {
perror("Fork failed");
exit(1);
}
if (pids[i] == 0) {
// Child process

```



```

close(pipefd[i][0]); // Tutup read end
double start_time = get_current_time();
// Eksekusi tugas berdasarkan index
switch (i) {
case 0:
task_sorting(array, ARRAY_SIZE);
strcpy(results[i].task_name, "Sorting Array");
break;
case 1:
task_searching(array, ARRAY_SIZE);
strcpy(results[i].task_name, "Searching Value");
break;
case 2:
task_calculation(array, ARRAY_SIZE);
strcpy(results[i].task_name, "Calculation Stats");
break;
}
double end_time = get_current_time();
// Isi data hasil
results[i].pid = getpid();
results[i].execution_time = end_time - start_time;
results[i].status = 0; // Success
// Kirim hasil ke parent melalui pipe
write(pipefd[i][1], &results[i], sizeof(ProcessResult));
close(pipefd[i][1]);
exit(0);
}
}

// Parent process - mengkoordinasi dan mengumpulkan hasil
printf("\n=== PARENT PROCESS MENUNGGU CHILD SELESAI ===\n");

```

```

for (int i = 0; i < NUM_PROCESSES; i++) {
close(pipefd[i][1]); // Tutup write end di parent

// Baca hasil dari pipe
ProcessResult result;
read(pipefd[i][0], &result, sizeof(ProcessResult));
results[i] = result;
close(pipefd[i][0]);

// Tunggu child process selesai
int status;
waitpid(pids[i], &status, 0);
if (WIFEXITED(status)) {
results[i].status = WEXITSTATUS(status);
}
}

// Tampilkan hasil summary
printf("\n=== HASIL EKSEKUSI SEMUA PROSES ===\n");
printf("%-10s %-20s %-12s %-8s\n",
"PID", "Task", "Time (s)", "Status");
printf("-----\n");

double total_time = 0;
for (int i = 0; i < NUM_PROCESSES; i++) {
printf("%-10d %-20s %-12.6f %-8s\n",
results[i].pid,
results[i].task_name,
results[i].execution_time,
results[i].status == 0 ? "Success" : "Failed");
total_time += results[i].execution_time;
}

printf("\n=== STATISTIK KESELURUHAN ===\n");
printf("Total processes: %d\n", NUM_PROCESSES);

```

```

printf("Total execution time: %.6f seconds\n", total_time);

printf("Parent Process PID: %d\n", getpid());

printf("Semua child processes telah selesai dieksekusi!\n");

return 0;

}

```

2.2 PRAKTIK TUGAS 2

2.2.1 Chat Application dengan IPC

Implementasi sistem chat room ini mendemonstrasikan konsep advanced inter-process communication melalui mekanisme shared memory architecture yang diintegrasikan dengan semaphore-based synchronization. Sistem ini memanfaatkan segment shared memory yang diakses secara konkuren oleh multiple processes (User1 dan User2) melalui key yang di-generate menggunakan ftok() dari file "chatfile". Setiap pesan dikemas dengan timestamp precision yang mencatat jam, menit, dan detik pengiriman, menciptakan audit trail yang komprehensif. Mekanisme circular buffer implementation dengan kapasitas 10 pesan mengimplementasikan strategi FIFO untuk memori terbatas, dimana pesan lama secara otomatis terdesak oleh pesan baru ketika buffer mencapai kapasitas maksimal. Semaphore operations dengan pola lock-unlock menjamin atomicity dalam critical section selama operasi baca-tulis, mencegah race conditions dan memastikan konsistensi data. Fungsi history menampilkan seluruh conversasi dalam format terstruktur, membuktikan efektivitas persistent communication channel antar proses yang independen. Arsitektur ini merepresentasikan pola producer-consumer dalam distributed systems, dimana setiap user bertindak sebagai kedua role secara simultan, menunjukkan robust-nya desain IPC dalam menangani komunikasi real-time antar proses yang berjalan pada environment yang terisolasi.

- **Buat file chat.c:**

```

rio_regex@DESKTOP-NVPMHQ:~$ nano chat.c
rio_regex@DESKTOP-NVPMHQ:~$ touch chatfile
rio_regex@DESKTOP-NVPMHQ:~$ gcc chat.c -o chat
rio_regex@DESKTOP-NVPMHQ:~$

```

- **Terminal 1:**

```

rio_regex@DESKTOP-NVPMHQ:~$ ./chat User1
=== CHAT ROOM ===
Welcome User1! (Type 'exit' to quit, 'history' to view messages)
User1> HALO BOSS
User1> BAGAIMANA KABARMU ?
User1> history

=== CHAT HISTORY ===
[22:43:58] User1: HALO BOSS
[22:44:50] User2: IYA BOSS?
[22:45:18] User1: BAGAIMANA KABARMU ?
[22:45:40] User2: BAIK BOSS
=====
User1> exit
Goodbye User1!
rio_regex@DESKTOP-NVPMHQ:~$

```

- **Terminal 2:**

```

rio_regex@DESKTOP-NVPMSHQ:~$ ./chat User2
=== CHAT ROOM ===
Welcome User2! (Type 'exit' to quit, 'history' to view messages)
User2> IYA BOSS?
User2> BAIK BOSS
User2> exit
Goodbye User2!
rio_regex@DESKTOP-NVPMSHQ:~$ |

```

2.2.2 Penjelasan Proses Kompilasi dan Eksekusi:

A. Persiapan Environment:

- **Pembuatan File Source Code** - Membuat file chat.c menggunakan text editor nano
- **File Key Generation** - Membuat file chatfile sebagai dasar pembuatan key IPC dengan ftok()
- **Library Integration** - Program mengintegrasikan multiple system libraries (sys/ipc.h, sys/shm.h, sys/sem.h) untuk IPC functionality

B. Proses Kompilasi:

- **Kompilasi Program** - Menjalankan gcc chat.c -o chat untuk mengkompilasi kode
- **Linking Process** - GCC menghubungkan object code dengan library sistem untuk:
 1. Shared memory operations (shmget, shmat, shmdt)
 2. Semaphore operations (semget, semop, semctl)
 3. System time functions (time, localtime)

C. Inisialisasi Sistem Chat:

- **Shared Memory Creation** - Membuat segment shared memory dengan shmget() menggunakan key dari ftok()
- **Memory Attachment** - Melakukan shmat() untuk mengattach shared memory ke address space proses
- **Semaphore Initialization** - Membuat dan menginisialisasi semaphore dengan nilai 1 untuk mutual exclusion
- **Struktur Data Setup** - Menginisialisasi chat_room structure dengan message_count = 0

D. Eksekusi Multi-User:

- **Launch User Sessions** - Menjalankan program di terminal terpisah:
 1. ./chat User1 di Terminal 1
 2. ./chat User2 di Terminal 2
- **User Authentication** - Sistem menerima username sebagai command-line argument
- **Interface Initialization** - Menampilkan welcome message dan command instructions

E. Operasi Chat Real-time:

- **Message Input** - User memasukkan pesan melalui prompt Username>

- **Command Processing** - Sistem memproses perintah khusus:
 1. history : Menampilkan semua pesan dalam buffer
 2. exit : Keluar dari aplikasi
- **Timestamp Generation** - Setiap pesan dilengkapi timestamp real-time
- **Semaphore Locking** - Menggunakan semaphore P() operation sebelum akses shared memory
- **Message Storage** - Menyimpan pesan ke circular buffer dengan strategi:
 1. Append jika buffer belum penuh
 2. Shift dan replace jika buffer penuh
- **Semaphore Unlocking** - Melakukan V() operation setelah operasi selesai

F. IPC Mechanism:

- **Concurrent Access Management** - Semaphore menjamin exclusive access ke shared memory
- **Data Consistency** - Mencegah race conditions selama read-write operations
- **Real-time Synchronization** - Perubahan data langsung terlihat oleh semua connected users

G. Process Termination:

- **Clean Exit** - User ketik exit untuk terminasi graceful
- **Memory Detachment** - Melakukan shmdt() untuk melepas shared memory
- **Resource Preservation** - Shared memory dan semaphore tetap aktif untuk user lainnya
- **Session Closure** - Menampilkan goodbye message dan mengakhiri proses

H. System Persistence:

- **Data Persistence** - Pesan tetap tersimpan selama shared memory aktif
- **Multi-session Support** - User dapat join/leave tanpa mempengaruhi data existing
- **Resource Management** - Shared memory hanya di-destroy ketika secara explicit dihapus

I. Kode c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/ipc.h>
```

```

#include <sys/shm.h>
#include <sys/sem.h>
#include <sys/types.h>
#include <time.h>

#define SHM_SIZE 1024
#define MAX_MESSAGES 10
#define MESSAGE_SIZE 100

// Struktur untuk shared memory
typedef struct {
    char messages[MAX_MESSAGES][MESSAGE_SIZE];
    int message_count;
    int read_count;
} chat_room;

// Union untuk semctl
union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

// Fungsi untuk mengirim pesan
void send_message(chat_room *room, int sem_id, const char *user, const char *message) {
    struct sembuf sem_op;

    // Lock semaphore
    sem_op.sem_num = 0;
    sem_op.sem_op = -1;
    sem_op.sem_flg = 0;
    semop(sem_id, &sem_op, 1);

    // Format pesan dengan timestamp
    time_t now = time(NULL);
    struct tm *t = localtime(&now);

```

```

char timestamp[20];
snprintf(timestamp, sizeof(timestamp), "%02d:%02d:%02d", t->tm_hour, t->tm_min, t
>tm_sec);
// Tambahkan pesan ke chat room
if (room->message_count < MAX_MESSAGES) {
    snprintf(room->messages[room->message_count], MESSAGE_SIZE,
    "[%s] %s: %s", timestamp, user, message);
    room->message_count++;
} else {
    // Geser pesan lama jika sudah penuh
    for (int i = 0; i < MAX_MESSAGES - 1; i++) {
        strcpy(room->messages[i], room->messages[i + 1]);
    }
    snprintf(room->messages[MAX_MESSAGES - 1], MESSAGE_SIZE,
    "[%s] %s: %s", timestamp, user, message);
}
// Unlock semaphore
sem_op.sem_num = 0;
sem_op.sem_op = 1;
sem_op.sem_flg = 0;
semop(sem_id, &sem_op, 1);
}
// Fungsi untuk menampilkan history chat
void show_history(chat_room *room, int sem_id) {
    struct sembuf sem_op;
    // Lock semaphore
    sem_op.sem_num = 0;
    sem_op.sem_op = -1;
    sem_op.sem_flg = 0;
    semop(sem_id, &sem_op, 1);

```

```

printf("\n=== CHAT HISTORY ===\n");
if (room->message_count == 0) {
printf("No messages yet.\n");
} else {
for (int i = 0; i < room->message_count; i++) {
printf("%s\n", room->messages[i]);
}
}
printf("=====\n");
// Unlock semaphore
sem_op.sem_num = 0;
sem_op.sem_op = 1;
sem_op.sem_flg = 0;
semop(sem_id, &sem_op, 1);
}
int main(int argc, char *argv[]) {
if (argc != 2) {
printf("Usage: %s <username>\n", argv[0]);
printf("Example: %s User1\n", argv[0]);
return 1;
}
char *username = argv[1];
key_t key = ftok("chatfile", 65);
int shm_id, sem_id;
chat_room *room;
union semun sem_arg;
// Buat shared memory
shm_id = shmget(key, sizeof(chat_room), 0666 | IPC_CREAT);
if (shm_id == -1) {
perror("shmget failed");

```



```

return 1;
}
// Attach shared memory
room = (chat_room*) shmat(shm_id, NULL, 0);
if (room == (void*) -1) {
perror("shmat failed");
return 1;
}
// Inisialisasi shared memory jika pertama kali
if (shmctl(shm_id, IPC_STAT, NULL) == 0) {
room->message_count = 0;
room->read_count = 0;
}
// Buat semaphore
sem_id = semget(key, 1, 0666 | IPC_CREAT);
if (sem_id == -1) {
perror("semget failed");
return 1;
}
// Inisialisasi semaphore
sem_arg.val = 1;
if (semctl(sem_id, 0, SETVAL, sem_arg) == -1) {
perror("semctl failed");
return 1;
}
printf("=== CHAT ROOM ===\n");
printf("Welcome %s! (Type 'exit' to quit, 'history' to view messages)\n", username);
char message[MESSAGE_SIZE];
while (1) {
printf("%s> ", username);

```

```

fflush(stdout);
if (fgets(message, sizeof(message), stdin) == NULL) {
break;
}
// Hapus newline
message[strcspn(message, "\n")] = 0;
// Periksa perintah
if (strcmp(message, "exit") == 0) {
break;
} else if (strcmp(message, "history") == 0) {
show_history(room, sem_id);
} else if (strlen(message) > 0) {
send_message(room, sem_id, username, message);
}
}
// Detach shared memory
shmdt(room);
printf("Goodbye %s!\n", username);
return 0;
}

```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa pemahaman mendalam tentang process management dan IPC merupakan komponen kritis dalam pengembangan sistem operasi modern. Implementasi sistem manajemen proses berhasil menunjukkan efisiensi parallel computing dengan waktu eksekusi total hanya 0.000289 detik, membuktikan optimalnya penggunaan fork(), pipe(), dan wait() untuk task distribution. Pada sisi IPC, aplikasi chat mengkonfirmasi efektivitas Shared Memory yang dikombinasikan dengan Semaphore untuk menjamin data consistency dalam environment konkuren. Terdapat trade-off signifikan antara mekanisme Pipes dan Shared Memory dimana Pipes menawarkan

keamanan melalui kernel mediation sementara Shared Memory memberikan performa superior untuk volume data besar. Pemilihan strategi IPC yang optimal harus mempertimbangkan faktor performa, kompleksitas implementasi, dan requirement keamanan dalam arsitektur distributed systems.